COMPREHENSIVE FIELD GUIDE

SelfOS — Live Demo & Setup Guide

Step-by-step guide to run and present a live demo of SelfOS with ChatGPT, Supabase, and the Web Dashboard

Author: **Abhradip** Repository: **https://github.com/Abhrxdip/SelfOS**

Phase 1: One-Time Cloud Setup (Takes 2 Minutes)

Step 1: Run the Database SQL Migration

Your Supabase project needs the core tables and vector indexes created:

1. Go to your **Supabase SQL Editor**: `https://supabase.com/dashboard/project/yxmcpuzjnzxwezurezty/sql/new`
2. Open `supabase/migrations/00000_all_in_one.sql` on your computer.
3. Copy the entire file and paste it into the SQL Editor.
4. Click **Run** (green button). *(This enables `pgvector` and creates `users`, `memories`, `agents`, `connections`, and `plans` tables).*

Step 2: Enable Supabase Auth & OAuth Server

1. In your Supabase Dashboard, go to **Authentication > Providers > Email**:
   - Ensure **Email** is toggled **ON**.
   - *Tip:* Disable "Confirm email" for instant test signups.
2. Go to **Authentication > OAuth Server**:
   - Turn **Enable OAuth Server** to **ON**.
   - Check **Allow Dynamic OAuth Apps**.

Phase 2: Launch the Project (Open 3 Terminals)

Terminal 1: Start the Public HTTPS Tunnel for ChatGPT

ChatGPT requires an HTTPS public endpoint to reach your local MCP server:

```
# Option A: using cloudflared (recommended, free, zero setup)
cloudflared tunnel --url http://localhost:3001

# Option B: using ngrok
ngrok http 3001
```

```
# Option C: using alpic
npx -y alpic tunnel --port 3001
```

Crucial Step: Copy your generated tunnel URL (e.g., `https://xxxx.trycloudflare.com`) and update `mcp-server-alpic/.env`:

```
MCP_SERVER_URL=https://xxxx.trycloudflare.com
```

Terminal 2: Start the MCP Server

```
cd c:\Users\abhra\OneDrive\Desktop\DigitalTwin\mcp-server-alpic
npm run dev
```

(Your MCP server starts on port 3001 with endpoint `/mcp`)

Terminal 3: Start the Web Dashboard

```
cd c:\Users\abhra\OneDrive\Desktop\DigitalTwin\webapp
npm run dev
```

(Your dashboard will be live at `http://localhost:3000`)

Phase 3: Connect MCP in ChatGPT

1. Open **https://chatgpt.com** and log into your account.
2. Navigate to **Settings > Connected Apps / Developer / MCP**.
3. Click **Add New MCP Server**:
   - **Name:** `SelfOS`
   - **Server URL:** `<YOUR_TUNNEL_URL>/mcp` *(e.g., `https://xxxx.trycloudflare.com/mcp`)*
4. Click **Connect**. Complete the Supabase login dialog when prompted to authorize the OAuth grant.

Phase 4: The Live Demo Script (What to Show & Say)

1. The Opening Pitch (30 seconds)

> *"Most AI assistants forget who you are as soon as a conversation ends. SelfOS is your personal AI operating system with a centralized memory layer. It lives across any AI client — you can teach it in ChatGPT, inspect it on the Web Dashboard, and collaborate with specialist agents."*

2. Scene 1: Autonomous Memory Layer (1 minute)

In **ChatGPT**, type naturally:

> *"Hi, remember that I am a Senior Full-Stack Engineer who specializes in Next.js and TypeScript, and I prefer async code reviews over long meetings."*

What happens behind the scenes:

- ChatGPT triggers SelfOS's `save-memory` tool.
- SelfOS creates a `text-embedding-ada-002` vector embedding and stores it in Supabase `pgvector`.
- **Show the audience:** Switch to `http://localhost:3000/twin/memories` on your dashboard to reveal the memory live and classified into categories!

3. Scene 2: Semantic Recall (30 seconds)

In **ChatGPT**, ask:

> *"What do you know about my tech stack and how I like to work?"*

What happens: ChatGPT executes `search-memories` via vector cosine similarity and recites back your exact working profile.

4. Scene 3: Specialist Agent Team & Interactive Widget (1 minute)

In **ChatGPT**, say:

> *"Show available specialist agents."*

- SelfOS invokes `list-agents` and renders an interactive visual widget directly in ChatGPT showing the 8 specialist agents.

Then say:

> *"Create a plan with the Frontend and Security agents to build an OAuth 2.1 authentication flow."*

- Claude specialist agents collaborate in parallel to generate actionable architecture.
- The plan is auto-saved. Show `http://localhost:3000/plans` to display the synchronized plan!